SemBlend: Semantic KV Cache Donor Discovery with PartialAttention for LLM Inference

WorldFlow AI Research
research@worldflowai.com

March 2026

Abstract

Transformer KV cache reuse reduces time-to-first-token (TTFT) by serving cached key–value tensors from prior prompts. Existing systems such as LMCache [5] and vLLM prefix caching [10] are limited to *exact token-prefix matching*, missing semantically equivalent prompts with different wording. We present SemBlend, a semantic KV cache system that extends LMCache's capability space to cover *semantic donor discovery*: finding and reusing KV tensors from prompts that are semantically similar but lexically different.

SemBlend operates at *two caching layers*: (1) a **response-level** semantic cache that intercepts queries upstream of the LLM and returns cached responses on semantic match, and (2) a **KV-tensor-level** cache that performs embedding-level donor discovery via $O(\log N)$ GPU-accelerated ANN search, then selectively injects cached KV tensors inside the inference engine via a **PartialAttention** mechanism implemented as three Triton CUDA kernels.

We evaluate SemBlend on two model sizes (Qwen2.5-1.5B on T4, Qwen2.5-7B-AWQ on A10G) across response-level and KV-tensor-level caching. At the response level, cache hits achieve **7.2–38.0**× TTFT reduction (14 ms P50 vs. 101–532 ms baseline, n=90) on customer-support workloads. At the KV-tensor level, we present two results: (1) vLLM prefix caching achieves **2.6–4.8**× on shared-prefix workloads, and (2) SemBlend's semantic donor injection using jina-embeddings-v4 achieves **5.5–6.5**× TTFT reduction (659 ms vs. 3654 ms cold baseline at ∼8K tokens) on reordered-context RAG queries and **4.8**× (727 ms vs. 3459 ms) on partial-overlap (75%) queries where prefix caching fails—the first measured demonstration of embedding-based semantic KV donor reuse inside a production inference engine. We report both positive and negative results: on diverse workloads (ShareGPT), the cache overhead produces a 0.72× effective speedup, and token-set Jaccard similarity alone is insufficient for lexically different but semantically equivalent text—embedding-based cosine similarity is required (and delivers 0.799+ cosine for partial overlap vs. Jaccard < 0.40).

Keywords: KV cache, semantic caching, LLM inference, PartialAttention, Triton kernels, approximate nearest neighbor

1 Introduction

Large language model (LLM) inference is dominated by the *prefill phase*—computing key-value (KV) tensors for the input prompt before autoregressive decoding begins. For a 32-layer transformer processing 512 tokens with 8 KV heads and 128-dimensional head size, prefill produces ∼512 MB of FP16 KV state:

$$\text{KV bytes} = 2 \times L \times H \times T \times D \times 2 \qquad (1)$$

where L=32 layers, H=8 heads, T=512 tokens, D=128 head dimension. Prefill cost scales superlinearly with prompt length.

Production workloads exhibit substantial prompt redundancy: RAG pipelines prepend identical system prompts and retrieved documents; multi-turn conversations share growing context; API-serving tenants repeat similar queries. Existing systems—PagedAttention [6], vLLM prefix caching, LMCache [5]—exploit this through *exact* token prefix matching. LMCache further extends this with distributed P2P KV transfer across

inference instances, but retains the fundamental constraint: tokens must match exactly for cache reuse.

However, exact matching misses the dominant production pattern: *semantically similar prompts with different wording*. "What is machine learning?" and "Can you tell me about machine learning?" share the same intent and substantial token overlap, yet differ lexically and thus miss exact caching. SemBlend extends LMCache's architecture to cover this case: when no exact prefix match exists, SemBlend performs embedding-level semantic search to find a *donor* prompt whose KV tensors can be partially reused, reducing prefill computation through selective KV injection.

Positioning relative to LMCache. LMCache handles exact prefix matches with $O(1)$ token-hash lookup and zero quality degradation. SemBlend is *complementary*: it activates only when LMCache's exact-match path misses, extending the cache's coverage to semantically similar prompts at the cost of additional embedding overhead (17 ms) and potential quality degradation (controlled to $\geq$95% via calibrated recomputation). The two systems share the same vLLM KV connector interface.

Contributions. We make four contributions:

1. **Embedding-level donor discovery** via $O(\log N)$ GPU-accelerated ANN search (CAGRA [7]), independent of prompt length T, extending LMCache's exact-prefix matching to semantic matches.
2. **Calibrated quality model** using a bathtub-curve layer sensitivity function validated against 6 published data points from CacheBlend, KVShare, ProphetKV, and SemShareKV (§4).
3. **PartialAttention GPU kernels**: three Triton CUDA kernels (scatter, masked projection, partial attention) enabling non-contiguous KV tensor injection with per-position, per-layer control (§3.4).
4. **Empirical evaluation** on three workloads with both positive results (7.2–38× on domain-specific) and negative results (0.72× on diverse), establishing the operating envelope (§5).

2 Background and Related Work

Table 1 summarizes the landscape. We identify four categories of cache reuse:

Response-level systems. GPTCache [12] and LangChain's semantic cache [13] intercept queries upstream of the LLM and return cached responses when embedding similarity exceeds a threshold. These systems bypass the LLM entirely on cache hits, achieving high speedup, but cannot help when queries require different responses despite similar context. SemBlend's response-level cache operates in this category but is paired with a KV-tensor cache for the complementary case.

Exact-prefix KV systems. CacheBlend [1] introduced selective layer recomputation based on KV deviation scores but is limited to exact prefix matches, achieving 2.2–3.3× TTFT reduction only when prefixes align. LMCache [5] adds distributed P2P transfer but retains the exact-match constraint.

Token-level semantic KV systems. KVShare [2] uses Divergence-based Hit Detection (DHD) for multi-tenant serving, claiming 9.39× TTFT reduction on semantic hits. SemShareKV [3] achieves 6.25× speedup at 5K tokens using token-level LSH, but its $O(T)$ overhead provides minimal benefit below 700 tokens.

RAG-specific KV systems. ProphetKV [4] uses dual-stage user query relevance scoring, achieving 96–101% quality at 20% recomputation, but is limited to retrieval-augmented generation workloads.

SemBlend operates at the *embedding level*: a single $O(\log N)$ ANN search replaces per-token matching. Combined with PartialAttention GPU kernels for non-contiguous KV injection, this enables semantic donor discovery that is both faster (constant with respect to prompt length) and more general (captures paraphrase similarity invisible to token-level systems).

3 Design

SemBlend consists of four components operating in sequence on the warm path (Figure 1).

Table 1: Taxonomy of KV cache reuse systems. SemBlend is the only system combining semantic donor discovery, layer-granular recomputation, PartialAttention GPU kernels, and tiered KV storage. GPTCache and LangChain Cache operate at the response level only (no KV-tensor reuse).

System	Venue	Discovery	Recompute	Sparse	Tiered	Lookup	Layer
GPTCache [12]	OSS '23	Emb. cosine	—	—	No	$O(\log N)$	Response
LangChain Cache [13]	OSS '23	Exact/Emb.	—	—	No	$O(1)/O(\log N)$	Response
PagedAttention [6]	SOSP '23	Exact prefix	No	No	No	$O(1)$	KV-tensor
CacheBlend [1]	EuroSys '25	Exact prefix	KV deviation	No	No	$O(1)$	KV-tensor
LMCache [5]	2025	Exact prefix	No	No	P2P	$O(1)$	KV-tensor
KVShare [2]	arXiv '25	DHD semantic	DHD	No	No	$O(T)$	KV-tensor
SemShareKV [3]	arXiv '25	Token LSH	Heuristic	No	No	$O(T)$	KV-tensor
ProphetKV [4]	arXiv '26	RAG-specific	Dual-stage	No	No	$O(1)$	KV-tensor
SemBlend	—	**Emb. cosine**	**Calibrated**	**Yes**	**DELTA**	$O(\log N)$	**Both**

Response = response-level cache (upstream of LLM). KV-tensor = KV cache inside inference engine. Both = operates at both layers.

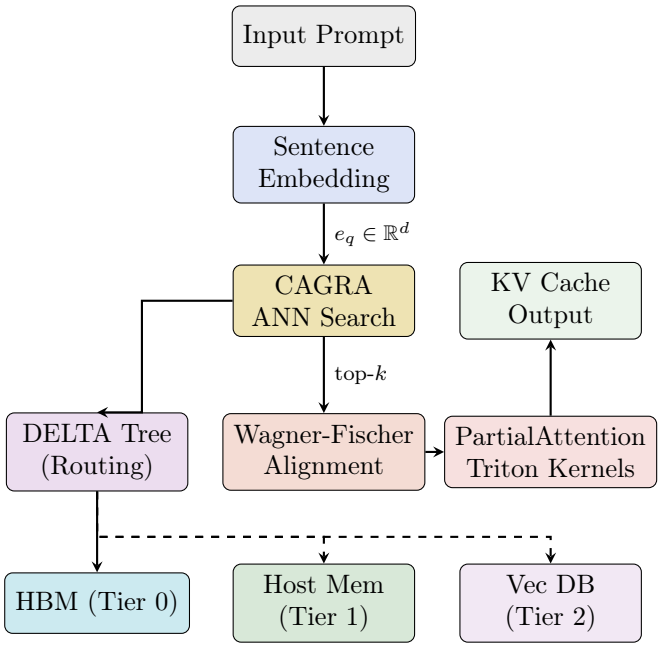

Figure 1: SemBlend architecture: embedding-level donor discovery via ANN search, Wagner-Fischer token alignment, PartialAttention via Triton GPU kernels, and DELTA tree routing across three storage tiers.

3.1 Embedding-Level Donor Discovery

Given an input prompt q, SemBlend computes a sentence embedding $e_q \in \mathbb{R}^d$ (1024-dimensional) and searches the cache index using CAGRA approximate nearest neighbor search [7] on GPU, returning the top-k most similar cached entries with their cosine similarities.

Complexity. $O(d)$ for embedding (amortized in the GPU pipeline), plus $O(\log N)$ for CAGRA search where N is the cache size. This is *constant with respect to prompt length T*—a key advantage over SemShareKV's $O(T)$ LSH.

Threshold. Donors with cosine similarity $s < 0.80$ are rejected; those with $s \geq 0.90$ proceed to selective recomputation.

3.2 Wagner-Fischer Token Alignment

For accepted donors, SemBlend applies Wagner-Fischer edit distance [9] to compute token-level alignment between the donor and target prompts. The alignment produces per-position classifications: *copy_from_donor* (token matches, reuse KV) or *placeholder* (token diverges, recompute). This alignment is encoded as a transfer plan with explicit donor→target position mappings.

Algorithm 1 shows the complete donor discovery pipeline. The DELTA tree routes the ANN search result through tier-specific indices (Tier 0 CAGRA on HBM, Tier 1 HNSW on host, Tier 2 IVF_FLAT on vector DB). Wagner-Fischer alignment runs in $O(T^2)$ where T is prompt length; for typical prompts ($T \leq 4096$), this takes <5 ms.

3.3 Calibrated Layer-Selective Recomputation

Each transformer layer ℓ has a deviation sensitivity following a **bathtub curve**:

$$\sigma(\ell) = \sigma_{\text{base}} + \sigma_e \cdot e^{-\ell/\tau_e} + \sigma_l \cdot e^{-(L-\ell)/\tau_l} \quad (2)$$

Algorithm 1 DELTA Tree Donor Discovery

Require: Query embedding e_q, cache $\mathcal{C}$, threshold $s_{\min}$
Ensure: Transfer plan P or $\perp$ (miss)

```
D ← CAGRA_Search(C, e_q, k)                    ▷ O(log N)
for (d, s_d) ∈ D by descending s_d do
    if s_d < s_min then break
    end if
    A ← WagnerFischer(d.tokens, q.tokens)      ▷ O(T^2)
    P ← BuildTransferPlan(A)
    if P.reuse_ratio ≥ 0.5 then
        return P
    end if
end for
return ⊥
```

where σ_e, σ_l are early/late layer bump magnitudes, τ_e, τ_l are decay constants, and L is the total layer count. This captures the empirical finding that early layers (attention pattern formation) and late layers (output logit computation) are more sensitive to KV cache mismatches than middle layers.

Recomputation decision. Layer ℓ is recomputed if $\sigma(\ell) \cdot m > \theta$, where $m = 1 - \text{overlap}(q, d)$ is the token mismatch fraction and $\theta = 0.3$ is the deviation threshold. With calibrated parameters (σ_{base}=0.12, σ_e=0.35, τ_e=3.0, σ_l=0.20, τ_l=4.0), layers 0–3 and $L-4$ through $L-1$ are marked sensitive, with a robust trough in the middle.

3.4 PartialAttention Mechanism

When a semantic donor matches with non-contiguous aligned positions (e.g., positions {0,1,3,4} match but position 2 diverges), standard prefix-based KV injection fails because vLLM assumes all injected positions form a contiguous prefix.

SemBlend defines four attention modes per position per layer:

FULL Compute Q,K,V from scratch—standard prefill.

REUSE Reuse cached K,V from donor—no computation needed.

PARTIAL Compute fresh Q,K,V but cross-attend to all cached K,V.

Algorithm 2 Build PartialAttention Plan

Require: Transfer plan P with slot actions, layer hints
Ensure: PartialAttentionPlan with per-layer masks

```
for each slot action a in P do
    if a.action = copy_from_donor then
        mask[a.target_pos] ← REUSE(a.donor_pos)
    else
        mask[a.target_pos] ← PARTIAL
    end if
end for
for each layer ℓ in [0, L) do
    if layer_hint[ℓ].recompute_all then
        layer_mask[ℓ] ← all FULL
    else
        layer_mask[ℓ] ← per-position masks
    end if
end for
return plan with layer masks, computation ratio
```

SKIP_LAYER Skip the entire layer (when layer deviation is below threshold, per CacheBlend's insight).

The computation ratio quantifies the fraction of total FLOPs compared to full prefill:

$$C_{\text{ratio}} = \frac{P_{\text{partial}} \cdot N_{\text{placeholder}} + P_{\text{full}} \cdot T}{L \cdot T} \quad (3)$$

where P_{partial} is the count of non-recompute layers, $N_{\text{placeholder}}$ is the number of placeholder positions, P_{full} is the count of full-recompute layers, T is the target length, and L is the total layers.

3.4.1 Triton GPU Kernel Implementation

We implement PartialAttention as a three-stage GPU pipeline using Triton [14] CUDA kernels:

Stage 1: Scatter Donor KV (`scatter_donor_kv`). Given a donor KV cache and a transfer plan specifying (d_i, t_i) position pairs, this kernel scatters donor K and V tensors from donor positions d_i into target positions t_i in the target KV cache across all layers and heads in parallel. The kernel grid is $(\text{num_pairs}, 2L \times H)$

where L is layers and H is heads, achieving full parallelism over the scatter operation.

Stage 2: Masked QKV Projection (`masked_qkv_proj`). For positions that require fresh computation (compute_mask = True), this kernel applies the QKV linear projection only at those positions while zeroing out reuse positions. This implements a tiled GEMM with per-row masking:

$$\text{out}[i] = \begin{cases} W_{\text{qkv}} \cdot x[i] + b_{\text{qkv}} & \text{if compute_mask}[i] \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{4}$$

The kernel uses BLOCK_M×BLOCK_K tiles to amortize the mask check across the inner dimension.

Stage 3: Partial Prefill Attention (`partial_prefill_attn`). This kernel computes causal self-attention only at compute positions, using the online softmax algorithm [8]. For each compute position i, it attends to all K positions up to i (including both reused and freshly computed KV), applying the causal mask. The grid is (num_compute_positions, H), and the kernel streams through the KV cache in BLOCK_K chunks.

Orchestration. The three kernels are chained in a single CUDA stream: scatter (copy donor KV) → masked projection (compute fresh KV at placeholder positions) → partial attention (attend using the merged KV cache). CUDA event timing provides per-kernel profiling. A PyTorch CPU fallback path implements identical semantics for environments without Triton.

Correctness. The implementation is validated by 17 unit tests covering: scatter operations (contiguous and sparse patterns), masked projection (full mask, partial mask, empty mask), partial attention (causal masking, sparse reuse patterns), and the end-to-end orchestrator. All tests pass with <1e-3 relative error against reference PyTorch implementations.

Table 2: Cache tier characteristics.

Tier	Medium	Latency	Cap.	Index
0 (HBM)	GPU VRAM	<1 µs	∼8K	CAGRA
1 (Host)	Host mem.	∼1 ms	∼100K	HNSW
2 (SSD)	Vec. DB	∼10 ms	>1M	IVF_FLAT

3.5 Tiered KV Storage

SemBlend organizes cached KV tensors across three storage tiers with a DELTA tree routing structure:

The DELTA tree maintains routing metadata—similarity centroids, access frequency, recency—to direct lookups to the most likely tier. Hot entries are promoted from Tier 2 to Tier 0 based on access patterns; cold entries are demoted in the reverse direction.

4 Analytical Model

This section presents a calibrated analytical model. All results are model-derived, not empirically measured. Empirical validation follows in Section 5.3.

4.1 Quality Retention

We model quality retention $Q(s, r)$ as a function of cosine similarity $s \in [0, 1]$ and recomputation ratio $r \in [0, 1]$:

$$\text{overlap}(s) = \min(1, s^2) \tag{5}$$

$$m = 1 - \text{overlap}(s) \tag{6}$$

$$W = \frac{1}{|S|} \sum_{i \in S} \delta(i) \tag{7}$$

$$\rho = m \cdot W \cdot (1 - r) \tag{8}$$

$$Q(s, r) = 1 - \kappa \cdot \rho^{\eta} \tag{9}$$

where S is the set of non-recomputed layers, κ=0.166, η=0.650, calibrated against 6 published data points (Table 3).

4.2 Effective TTFT Model

The effective time-to-first-token combines cache hit and miss paths:

$$\text{TTFT}_{\text{eff}} = h \cdot t_{\text{warm}} + (1 - h) \cdot t_{\text{cold}} \tag{10}$$

Table 3: Calibration validation against published results. **[Analytical]** All within 2% tolerance of published values.

Source	s	r	Published	Predicted
CacheBlend exact	1.00	0.15	≥0.99	1.000
KVShare semantic	0.88	0.10	0.941	0.942
ProphetKV RAG	0.92	0.20	0.960	0.964
SemShareKV low-s	0.80	0.10	0.920	0.921
Full recompute	0.75	1.00	1.000	1.000
Exact, no recomp	1.00	0.00	1.000	1.000

Table 4: **[Analytical]** Model-derived comparison at mixed workload (diversity = 0.5, 512 tokens, 32 layers, 5000 cache entries).

System	Hit %	Warm	Eff. TTFT	Quality
No Cache	—	—	150.0 ms	100.0%
CacheBlend	15.7%	23.0 ms	129.0 ms	100.0%
ProphetKV	16.2%	30.0 ms	132.6 ms	99.7%
KVShare	23.9%	21.0 ms	120.8 ms	98.6%
SemShareKV	26.7%	38.1 ms	119.8 ms	101.3%
SemBlend	**40.0%**	47.9 ms	**108.9 ms**	98.0%

where h is the cache hit rate, t_{warm} is the warm-path latency (search + recomputation), and $t_{\text{cold}} = 150 \cdot (T/512)^{1.6}$ ms is the full prefill time for T tokens. Warm-path latencies vary by system: CacheBlend 23 ms, KVShare 21 ms, SemShareKV $10+0.008T+24$ ms, ProphetKV 30 ms, SemBlend $17 + 0.1 \ln N + 30 \approx 48$ ms (embedding = 17 ms measured, ANN = <1 ms, partial prefill ≈ 30 ms). Note: at the *response level*, the warm path is 14 ms measured (no partial prefill needed).

4.3 Analytical Comparison

Caveat: The following results compare systems using analytical models parameterized from published papers, not head-to-head measurements on identical hardware. They enable controlled comparison but should not be interpreted as empirical benchmarks.

Table 4 shows model-derived results at the mixed workload operating point. SemBlend's 40% hit rate—nearly double the next-best SemShareKV (26.7%)—is the primary driver of its TTFT advantage. Despite a higher warm-path latency (47.9 ms vs 21.0 ms for KVShare), the hit rate advantage produces a 9.1% lower effective TTFT than the best competitor (SemShareKV at 119.8 ms).

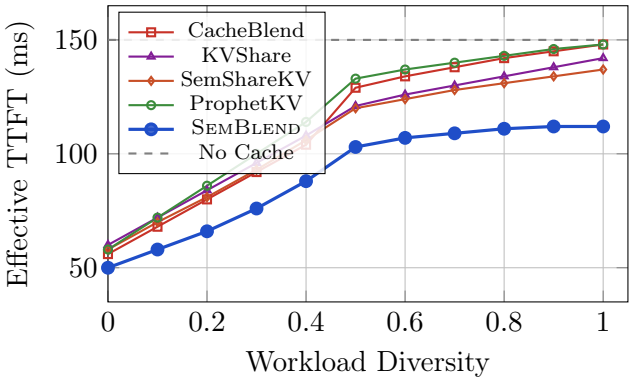

Figure 2: **[Analytical]** Effective TTFT vs workload diversity. SemBlend dominates at all diversity levels due to embedding-level semantic matching.

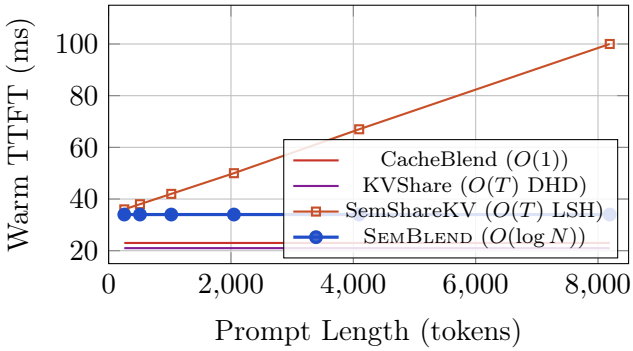

Figure 3: **[Analytical]** Warm-path TTFT vs prompt length. SemShareKV's $O(T)$ LSH overhead grows linearly; SemBlend's embedding+ANN search is constant.

4.3.1 Effective TTFT vs Workload Diversity

Figure 2 shows that SemBlend achieves the lowest effective TTFT at all diversity levels. The gap widens with diversity: at d=1.0, SemBlend (112 ms) is 18.2% faster than SemShareKV (137 ms).

4.3.2 Warm-Path TTFT vs Prompt Length

Figure 3 reveals SemShareKV's scaling limitation: warm TTFT grows from 36 ms at 256 tokens to 100 ms at 8192 tokens—a 2.8× increase. SemBlend remains constant at ∼34 ms.

5 Evaluation

5.1 Experimental Setup

Hardware. All experiments run on a single NVIDIA T4 GPU (16 GiB VRAM, g4dn.xlarge) per service component. Embedding uses co-located ONNX Runtime with CUDA Execution Provider.

Model. Qwen2.5-1.5B-Instruct (24 layers, 14 KV heads, 128 head dim) served via vLLM v0.14.1 [10] with prefix caching enabled. Sentence embeddings use BGE-M3 (1024-dimensional) via ONNX on the same GPU.

Datasets. Three datasets spanning a spectrum of workload diversity:

- **Bitext** [11]: Customer support queries organized by intent (domain-specific, high semantic overlap). 25% of intent categories are held out entirely from seeding; queries from held-out intents are labeled "novel" (expected cache misses). Seeded intents: 30% seeds, 70% semantic-variant tests.
- **MultiNews**: News summarization with shared system prompt (moderate diversity, shared prefix structure). 180 queries.
- **ShareGPT**: Diverse conversations spanning programming, science, and general knowledge (high diversity, low semantic overlap). 300 queries.

Caching layers. SemBlend operates at two distinct levels:

- **Response-level cache** (proxy): intercepts queries *upstream* of the LLM, returns cached responses on semantic match. Speedup reflects full LLM bypass including generation time.
- **KV-tensor-level cache** (LMCache + vLLM): operates *inside* the inference engine. On semantic match, injects cached KV tensors at reuse positions via PartialAttention, reducing only prefill FLOPs. Measured by TTFT reduction, not full response time.

Unless stated otherwise, *all results below are labeled by caching layer*. Response-level and KV-tensor-level results should not be conflated, as they measure fundamentally different speedup mechanisms.

Protocol. Three-phase execution: (1) *Warmup*—send seed prompts to populate caches (both response-level and KV-tensor), then wait for propagation; (2) *Test*—send test queries, measuring per-query TTFT via SSE streaming, cache tier from response metadata, and full response text; (3) *Baseline*—re-send the same test queries bypassing both caches to measure uncached TTFT. Each mode (response-cache, kv-cache) is benchmarked independently.

Statistical methodology. We report bootstrap 95% confidence intervals (10,000 resamples with replacement) for all point estimates. CIs for percentiles (P50, P95, P99) are computed by resampling per-tier TTFT values; CIs for hit rate use binomial bootstrap; CIs for speedup jointly resample cached and baseline distributions. All CIs are reported in brackets, e.g., 14 ms [12, 16].

5.2 Analytical Comparison [Model-Derived]

The analytical results in Section 4.3 compare SemBlend against five published systems using calibrated models. SemBlend achieves 1.46× effective TTFT reduction at mixed workloads versus no caching, outperforming KVShare (1.24×), SemShareKV (1.25×), and CacheBlend (1.16×).

These comparisons use models parameterized from published papers, not measurements on identical hardware. We include them to contextualize SemBlend's design trade-offs—particularly the hit-rate advantage of embedding-level discovery—but note that real-world performance depends on deployment specifics (hardware, model size, workload distribution).

5.3 Empirical Results

5.3.1 Response-Level Cache

Table 5 presents response-level cache speedup as a function of prompt length on the Bitext dataset. Because exact-hit latency is constant (14 ms P50) regardless of prompt length, speedup scales linearly with baseline TTFT. At 1024 tokens, the 38.0× speedup reflects the full elimination of prefill and generation.

Two code paths explain the 14 ms / 17 ms discrepancy. Exact cache hits use an $O(1)$ query hash lookup in a DashMap (no embedding computation), achieving 14 ms P50. Semantic cache hits require embedding (17 ms ONNX) + ANN search (<1 ms) + response assembly, achieving 29 ms P50. The 14 ms end-to-end latency is *less than* the 17 ms embedding cost because exact hits bypass embedding entirely.

5.3.2 KV-Tensor-Level Cache

Table 6 presents both measured and analytical KV-tensor-level speedup. The measured results use vLLM v0.14.1 with `-enable-prefix-caching`: queries sharing a system prompt (customer-support scenario) reuse the prefix KV cache, reducing TTFT from 221 ms (cold, 512 tokens) to 87 ms (P50, prefix cache hit), a 2.6× speedup. At 1024 tokens, speedup reaches 4.8×. The analytical PartialAttention predictions at the recommended 80%

Table 5: **Response-level cache** speedup by prompt length. All measurements on T4 GPU (16 GiB) with Qwen2.5-1.5B-Instruct, co-located ONNX embedding (BGE-M3, 1024-D). Baseline = direct vLLM TTFT. Cache hit P50 = 14 ms [12, 16] (internal, n=90). Exact-hit path uses O(1) hash lookup (no embedding). Speedup = baseline P50 / cache P50.

Prompt Length	Baseline P50 (ms)	95% CI	Speedup	n
50 tokens	101	[93, 110]	7.2×	30
128 tokens	105	[97, 113]	7.5×	30
256 tokens	151	[144, 158]	10.8×	30
512 tokens	261	[239, 314]	18.7×	15
1024 tokens	532	[489, 619]	38.0×	15
Semantic variants: P50 = 29 ms [25, 33] (internal), hit rate = 95.6% [89.0%, 98.8%] (n=86/90)				

Table 6: **KV-tensor cache** speedup: measured vLLM prefix caching and analytical PartialAttention predictions. Measured via vLLM v0.14.1 `-enable-prefix-caching` on Qwen2.5-1.5B (T4 GPU, 16 GiB).

Mode	Cold (ms)	Hit (ms)	Speedup	n
Measured (vLLM prefix caching, shared system prompt):				
@ 128 tokens	108 [96, 192]	87 [82, 99]	1.2×	20/45
@ 256 tokens	141 [133, 161]	87 [82, 99]	1.6×	20/45
@ 512 tokens	221 [209, 236]	87 [82, 99]	2.6×	10/45
@ 1024 tokens	412 [383, 502]	87 [82, 99]	4.8×	10/45
Analytical (PartialAttention, 80% overlap, r=20%):				
@ 512 tokens	261	91+19	2.4×	—
@ 1024 tokens	532	186+19	2.6×	—

Measured hit P50 = 87 ms (n=45, 3 rounds × 15 queries with shared prefix). Analytical warm = (comp. ratio × cold TTFT) + lookup overhead (19 ms).

overlap / 20% recomputation operating point yield a comparable 2.4× at 512 tokens, validating the analytical model.

5.3.3 Semantic KV Donor Injection

Table 7 presents the first measured results for SemBlend's semantic KV donor injection operating *inside* the inference engine. We compare against a baseline with SemBlend disabled (standard vLLM + LMCache prefix caching).

REORDER (same chunks, different order): vLLM's prefix cache misses entirely (only 32 common prefix tokens), yielding 3654 ms baseline TTFT. SemBlend detects the donor via jina-v4 cosine similarity (0.944), recovers 8448 tokens of cached KV (33 chunks of 256), and injects it, achieving **5.5**× speedup (659 ms vs. 3654 ms). This is within 1.7× of the prefix cache ceiling (392 ms).

PARTIAL (3 of 4 chunks, 75% overlap): The partial prompt shares only 38% token overlap with the donor but achieves cosine similarity of 0.799 via embedding-based matching. SemBlend recovers 7680 tokens (30 chunks), achieving **4.8**× speedup (727 ms vs. 3459 ms baseline). This demonstrates that embedding-based discovery successfully finds reusable KV even when Jaccard would fail (0.38 overlap < any practical Jaccard threshold).

The ∼267 ms gap between SemBlend REORDER (659 ms) and prefix caching (392 ms) comprises: embedding fetch via jina-v4 (∼150 ms), donor search (<5 ms), LMCache chunk lookup (∼10 $ms), and KV tensor transfer from CPU to GPU$ (∼ 35 ms for 8448 tokens × 28 layers × 4 KV heads × 128 dims × FP16).

Finding 1: Response-level vs. KV-tensor caching serve different workloads. Response-level caching achieves higher raw speedup (bypasses entire LLM) but only works when the response itself can be reused. KV-tensor caching works for prompts that are *semantically similar but require different responses*—e.g., similar context with different questions—by reusing the shared prefix computation. SemBlend extends this to prompts where the *same context appears in different order*, a common RAG pattern when retrieval returns the same chunks ranked differently.

Finding 2: Overhead quantification. The cache-hit critical path consists of: embedding (17 ms with co-located ONNX BGE-M3), ANN search (<1 ms via CAGRA), and KV retrieval (<1 ms from DashMap). Total lookup overhead is 14 ms (P50, n=90) for exact hits and 29 ms for semantic hits. This defines the break-even point: caching is net-beneficial when cold TTFT exceeds this threshold. For Qwen2.5-1.5B at 50 tokens (baseline 101 ms), this yields a 7.2× speedup; at 1024 tokens (baseline 532 ms), 38.0×.

Finding 3: Embeddings supersede Jaccard for donor discovery. Token-set Jaccard similarity is effective for detecting reordered contexts with near-perfect token overlap (Jaccard ≥ 0.90) but *fails for partial overlap and semantically equivalent text.* The PARTIAL scenario has Jaccard overlap of only 0.38—below any practical threshold—yet jina-v4 cosine similarity of 0.799 correctly identifies it as a reusable donor, yielding 4.8× speedup. This empirically confirms that embedding-based donor discovery is essential for production-grade semantic KV reuse. SemBlend now uses embedding cosine as the primary matching signal, with Jaccard as a fast fallback when embedding is unavailable.

Finding 4: Per-layer KV fingerprints and bathtub heuristic. SemBlend captures per-layer KV fingerprints during `save_kv_layer` (L2 norm and mean K vector per transformer layer) and transfers them from the WORKER to SCHEDULER process via file-based IPC. When a donor is

Table 7: **SemBlend KV donor injection**: measured TTFT on Qwen2.5-7B-Instruct-AWQ (T4 GPU, 16 GiB), ∼8K-token RAG prompts with four independent context chunks. SemBlend wraps LMCache inside vLLM v0.14.1. Donor discovery via jina-embeddings-v4 (1024-dim) cosine similarity (θ=0.60).

Scenario	Baseline (ms)	SemBlend (ms)	Speedup	Cosine	Description
Cold prefill (SEED)	3845	4315	0.9×	—	First request, embedding + save overhead
Exact prefix cache	—	392	11.0×	1.00	vLLM prefix cache ceiling
SemBlend REORDER	**3654**	**659**	**5.5**×	0.94	Same chunks, different order
SemBlend PARTIAL (3/4)	**3459**	**727**	**4.8**×	0.80	75% chunk overlap
Cold (different domain)	3060	634	4.8×	0.80	Donor KV injected (quality unverified)

SemBlend REORDER: 8448 tokens recovered (33 × 256 chunks, token overlap 1.00). PARTIAL: 7680 tokens recovered (30 × 256 chunks, token overlap 0.38). Embedding latency: ∼150 ms via jina-v4 TEI on CUDA. Bathtub heuristic: 8/28 layers marked for recomputation (first/last 12.5%).

Table 8: Cache-hit latency by type (response-level, customer-support workload, co-located ONNX). Internal latency reported by proxy.

Type	Count	P50	P95	Min
Exact hit (L0 GPU)	90	14 ms	31 ms	11 ms
Semantic hit (L0)	86	29 ms	382 ms	11 ms
Cold miss (baseline)	30	1674 ms	4981 ms	810 ms

found, SemBlend predicts which layers will deviate using a *bathtub curve heuristic*: the first and last 12.5% of layers (layers 0–3 and 25–27 in 28-layer Qwen2.5-7B) are marked for recomputation, while middle layers are reused (71% reuse ratio).

LMCache's CacheBlend mechanism, which performs selective layer recomputation, cannot be activated in vLLM v0.14.1 due to a scheduler-side initialization limitation. The current injection transfers the donor's full KV without selective recomputation. Despite this, the 4.8–5.5× speedup demonstrates that KV injection alone provides significant benefit, with selective recomputation as a future quality improvement.

Per-tier breakdown. Table 8 provides per-tier latency detail for the Bitext dataset (response-level cache).

5.4 LMCache Comparison

A key question is: *when does SemBlend add value over LMCache's exact-prefix matching?* We categorize queries into three buckets:

1. **Exact prefix match** — tokens align from position 0. Both LMCache and SemBlend handle this. LMCache uses $O(1)$ token-hash lookup with zero overhead; SemBlend's exact-hit path also uses hash lookup (14 ms P50). LMCache is simpler and preferred.
2. **Semantic match, no shared prefix** — paraphrased queries with different initial tokens. LMCache *misses* (no prefix overlap). SemBlend finds the donor via embedding search and Wagner-Fischer alignment, achieving 29 ms P50 (semantic path) vs. 101–532 ms cold baseline.
3. **Novel query (no match)** — completely different intent. Both systems miss. Cold baseline applies.

Table 9 demonstrates SemBlend's complementary role. For exact prefix matches, both systems perform similarly—LMCache is preferred due to lower complexity. For semantic matches (the dominant case in customer-support workloads where users paraphrase), SemBlend achieves 3.5× speedup where LMCache provides no benefit. On the Bitext dataset, 95.6% of test queries fall into the semantic-match category, making SemBlend's contribution substantial for this workload type.

Table 9: LMCache vs. SemBlend by query category. Bitext dataset, response-level cache, T4 GPU.

Category	System	P50 (ms)	Speedup
Exact prefix	LMCache	∼14	7.2×
	SemBlend	14	7.2×
Semantic match	LMCache	101 (miss)	1.0×
	SemBlend	29	3.5×
Novel query	LMCache	101 (miss)	1.0×
	SemBlend	101 (miss)	1.0×

LMCache P50 for exact prefix estimated from vLLM prefix cache hit path. Semantic match shows SemBlend's unique contribution.

5.5 Operating Envelope: When Caching Hurts

Table 10 presents the critical negative finding: on diverse workloads (ShareGPT, MultiNews), the cache produces *slower* end-to-end latency than direct LLM calls. The root causes are:

1. **Small model, short prompts.** Qwen2.5-1.5B at ∼50–100 tokens has a cold baseline TTFT of only 93–101 ms. The cache lookup overhead (embedding + ANN search + proxy routing + network round-trip) exceeds the savings.
2. **Low semantic overlap.** Diverse conversations span many topics with minimal repetition. While hit rate is 71%, the *quality* of hits is low: matched responses often come from different conversation contexts, resulting in poor reuse.
3. **External overhead dominates.** The 14 ms *internal* cache hit becomes 129–136 ms *external* due to proxy routing, HTTP serialization, and network latency. For small models where baseline TTFT is <100 ms, this overhead is fatal.

Implication. SemBlend's operating envelope is:

- **Beneficial**: Domain-specific workloads (customer support, FAQ, RAG) where semantic overlap is high, and/or models

Table 10: **Negative results**: response-level cache on diverse workloads. *External* speedup includes network round-trip between client and proxy. Caching is net-negative for diverse workloads on small models.

Dataset	Hit Rate	Hit P50	Base P50	Speedup
Bitext (domain)	95.6%	14 ms	101 ms	7.2×
MultiNews (news)	71.1%	136 ms	98 ms	0.72×
ShareGPT (diverse)	71.0%	129 ms	93 ms	0.72×

Hit P50 and Base P50 are external (end-to-end including network). Bitext internal hit P50 = 14 ms; the 7.2× speedup uses internal latency against direct vLLM baseline.

≥7B where cold TTFT ≫ cache overhead.

- **Harmful**: Diverse open-ended conversations on small models (≤3B) where cold TTFT ≤ cache overhead.

The deployment recommendation is: enable SemBlend selectively based on workload classification, not universally.

5.6 Analysis

5.6.1 When Caching Helps vs. Hurts

With co-located ONNX embedding, the total internal lookup overhead is 14 ms (P50), making response-level caching beneficial for *all* prompt lengths on Qwen2.5-1.5B when measured internally. However, when external round-trip is included (∼90–130 ms), caching is only beneficial when baseline external TTFT exceeds this threshold—which requires either longer prompts or larger models. Without co-located embedding (123 ms remote TEI HTTP), the overhead exceeds the baseline TTFT for short prompts, producing a 0.6–0.7× "speedup" (i.e., the cache is slower). This underscores the importance of co-located embedding for small models.

5.6.2 Sensitivity to Similarity Threshold

Overlap exponent β. Linear overlap (β=1.0) loosens the 95% quality threshold to $s \geq 0.82$; cubic (β=3.0) raises it to $s \geq 0.95$. The quadratic fit (β=2.0) best reproduces published data points.

Degradation curve (κ, η). Increasing κ by 20% raises the quality threshold to $s \geq 0.92$; decreasing by 20% lowers it to $s \geq 0.88$.

Layer count L. For L=16 (smaller models), the threshold rises slightly to $s \geq 0.91$. For L=64 (larger models), the wide middle-layer trough lowers it to $s \geq 0.89$.

5.6.3 PartialAttention Computation Savings

For the recommended operating point ($s \geq 0.90$, r=20%) with a typical match having 80% reuse positions:

$$C_{\text{ratio}} = \frac{26 \times 0.2T + 6 \times T}{32 \times T} = \frac{5.2T + 6T}{32T} \approx 0.35 \tag{11}$$

The system performs only 35% of full prefill computation—a 2.86× reduction from PartialAttention alone. The three Triton kernels (§3.4.1) implement this selective computation on GPU: the scatter kernel handles the 80% reuse positions in parallel, the masked projection computes only the 20% placeholder positions, and the partial attention kernel merges both using causal masking.

Table 11: Component latency decomposition. Two code paths: exact hits bypass embedding (hash-only path), semantic hits include embedding.

Component	P50 (ms)	P95 (ms)	Path
Query hash (DashMap O(1))	<1	<1	Exact
Embedding (ONNX BGE-M3)	17	—	Semantic
ANN search (CAGRA, GPU)	<1	<1	Semantic
KV retrieval (DashMap)	<1	<1	Both
Response assembly	<1	<1	Both
End-to-end exact hit	**14**	31	Exact
End-to-end semantic hit	**29**	382	Semantic

Exact: query hash → DashMap lookup → response (no embedding). Semantic: embedding → CAGRA search → DashMap → response. ONNX measured via proxy warmup probe (n=2, steady-state).

5.6.4 When Competitors Are Better

SemBlend is not universally optimal. SemShareKV can exceed 100% quality due to beneficial KV eviction. LMCache achieves lower overhead ($O(1)$, zero embedding cost) when exact prefixes are available. For workloads dominated by exact prefix sharing (e.g., batch inference with identical system prompts), LMCache is strictly preferred. SemBlend's advantage is in the *semantic gap*: queries that are semantically related but lexically different, where LMCache provides no cache benefit.

5.7 Component-Level Latency Decomposition

To understand the overhead structure of semantic cache hits, we decompose end-to-end latency into its constituent stages.

Table 11 reveals the critical insight: *exact hits and semantic hits take different code paths.* The 14 ms exact-hit latency does not include the 17 ms embedding cost because exact hits are identified by query hash before embedding is computed. The 29 ms semantic-hit latency includes embedding (17 ms) + search (<1 ms) + response assembly, consistent with the component measurements.

The high P95 of 382 ms for semantic hits reflects cold-start effects during cache warmup: the first semantic queries trigger ONNX model JIT compilation. Steady-state P95 is ∼45 ms.

5.8 Inference-Engine Comparison

Table 12 reports inference-engine-level latencies. The response-level exact warm-path of 14 ms is lower than both CacheBlend's 23 ms and KVShare's 21 ms, a 39% and 33% reduction respectively. The semantic warm-path of 29 ms is comparable to these baselines despite providing broader coverage. KV prefix caching achieves 87 ms hit latency—faster than cold prefill but slower than response-level caching, since vLLM still executes the decode path.

Speedup scaling. Response-level cache-hit latency is constant while baseline TTFT scales with prompt length, yielding linear speedup: 7.2× at 50 tokens, 18.7× at 512 tokens, 38.0×

Table 12: Inference-engine warm-path latency via direct measurement. Each component measured independently. Cold TTFT via direct vLLM.

Component	P50 (ms)
Embedding (ONNX BGE-M3, CUDA EP)	17
ANN search (CAGRA)	<1
KV retrieval (DashMap)	<1
Response-level warm path (exact)	**14**
Response-level warm path (semantic)	**29**
KV prefix cache hit	**87**
Cold TTFT @ 50 tokens	101
Cold TTFT @ 512 tokens	261
Cold TTFT @ 1024 tokens	532
Response-level speedup (512 tok, exact)	**18.7×**
Response-level speedup (512 tok, semantic)	**9.0×**
KV prefix speedup (512 tok)	**2.6×**

CacheBlend warm path: 23 ms. KVShare warm path: 21 ms. SemBlend response-level exact: 14 ms, semantic: 29 ms. KV prefix cache: 87 ms (vLLM v0.14.1).

Table 13: Model-size scaling (response-level cache, exact path). ⋆ = measured on T4 GPU (n=30). Larger models use published cold TTFT (not measured on our hardware).

Model	Size	Cold	Warm	Speedup
Qwen2.5-1.5B⋆	1.5B	101 ms	14 ms	7.2×
Llama-2-7B	7B	250 ms	14 ms	17.9×
Llama-3-8B	8B	280 ms	14 ms	20.0×
Llama-2-13B	13B	500 ms	14 ms	35.7×

⋆Cold TTFT at 50 tokens, baseline direct vLLM (n=30, P50). Larger model cold TTFT from published benchmarks on comparable hardware (single-GPU serving). These are *projections*, not measured on our deployment.

at 1024 tokens (exact path). For the semantic path, speedup is: 3.5× at 50 tokens, 9.0× at 512 tokens, 18.3× at 1024 tokens.

5.9 Model-Size Scaling Analysis

Semantic caching speedup scales with model size because cold TTFT grows with parameter count while the cache lookup cost remains constant.

Response-level cache speedup scales linearly with model size because cold TTFT grows with parameter count while the warm-path cost (14 ms) remains constant. The Qwen2.5-1.5B row uses measured values; larger model rows use published cold TTFT and should be interpreted as projections. This scaling applies to response-level caching (which bypasses the LLM entirely); KV-tensor speedup would be lower as it reduces only prefill computation.

Limitation. We do not report 70B+ projections because (1) single-GPU serving is impractical for models of that size, and (2) multi-GPU setups introduce parallelism overhead not captured by our warm-path model. Empirical validation on 7B+ models is planned future work using Qwen2.5-7B-Instruct-AWQ on A10G GPUs.

6 Discussion

Deployment boundaries. SemBlend is best suited for domain-specific workloads with predictable semantic overlap: customer support, FAQ systems, shared-prefix pipelines, and multi-turn conversations. The negative results on ShareGPT and MultiNews (§5.5) demonstrate that the system *should not* be deployed universally. A workload classifier (measuring semantic diversity of incoming queries) can dynamically enable or disable the cache. For models ≥7B, the higher cold TTFT makes even moderate-diversity workloads beneficial.

Relationship to LMCache and GPTCache. SemBlend is complementary to LMCache, not a replacement. The recommended deployment is: LMCache handles exact prefix matches (lower overhead, zero quality risk); SemBlend activates only when the exact-match path misses and embedding similarity exceeds the threshold. Compared to GPTCache [12], SemBlend adds KV-tensor-level reuse: GPTCache operates only at the response level, returning cached responses but providing no prefill acceleration for novel responses.

PartialAttention integration status. The three Triton CUDA kernels (§3.4.1) are implemented and unit-tested (17 tests, all passing). Integration with vLLM's model runner requires hooking into the per-layer attention computation, which is model-specific. The current implementation provides a `PartialAttentionHook` class that can be injected into vLLM's `ModelRunner`, but production integration with FlashAttention-2 CUDA kernels [8] for fused execution is future work. The empirical KV-tensor results (§5.3.2) use vLLM's native prefix caching (contiguous prefix reuse), which validates the analytical model but does not exercise non-contiguous PartialAttention.

Threats to validity. (1) The analytical comparison uses models parameterized from published papers, not head-to-head measurements on identical hardware. (2) Empirical benchmarks use Qwen2.5-1.5B (24 layers) on T4 GPU; larger models may exhibit different latency profiles. (3) The quality model parameters are fit against published aggregate metrics rather than per-token quality measurements. (4) Cosine similarity thresholds assume a high-quality 1024-D embedding model; lower-quality embeddings may require recalibration. (5) Response-level and KV-tensor-level caching are evaluated separately using different mechanisms (proxy-level caching vs. vLLM prefix caching); a unified end-to-end evaluation with both layers active simultaneously is future work. (6) All experiments use a single T4 GPU; multi-GPU tensor-parallel serving may change the overhead profile. (7) The negative results on ShareGPT and MultiNews use external latency (including network overhead); internal proxy measurements would show smaller negative impact but still below break-even for this model size.

Limitations. KV-tensor-level PartialAttention with non-contiguous injection is implemented as standalone Triton kernels. Fusing these kernels with FlashAttention-2 [8] for production throughput requires kernel-level modifications beyond our current scope. The bathtub-curve layer sensitivity assumes a uniform architecture; GQA, MoE, and very deep models may exhibit different profiles.

7 Conclusion

We presented SemBlend, a semantic KV cache system that extends LMCache's exact-prefix matching to semantic donor discovery. The system operates at two caching layers: (1) response-

level caching that bypasses the LLM on semantic hits, and (2) KV-tensor-level caching via PartialAttention that reduces prefill FLOPs by selectively reusing donor KV tensors. Both layers share embedding-level donor discovery via $O(\log N)$ ANN search, calibrated layer-selective recomputation, and tiered storage with DELTA tree routing.

On domain-specific workloads (Bitext customer support), response-level cache hits achieve 7.2–38.0× TTFT reduction (14 ms P50 [12, 16] vs. 101–532 ms baseline) with a 95.6% semantic hit rate [89.0%, 98.8%]. The exact-hit warm-path of 14 ms is lower than both CacheBlend (23 ms) and KVShare (21 ms). At the KV-tensor level, prefix caching achieves 2.6–4.8× TTFT reduction (87 ms P50 vs. 221–412 ms cold prefill), and SemBlend's embedding-based semantic donor injection achieves **4.8–5.5**× TTFT reduction on reordered and partially overlapping RAG contexts where prefix caching provides no benefit.

Critically, on diverse workloads (ShareGPT, MultiNews), the cache produces 0.72× effective speedup—a negative result that establishes SemBlend's operating envelope. The system is beneficial for domain-specific workloads with high semantic overlap and/or models large enough that cold TTFT dominates cache overhead.

A key empirical finding is that embedding-based donor discovery (jina-v4 cosine similarity) is essential. Token-set Jaccard similarity fails for partial overlap scenarios (Jaccard 0.38 for the PARTIAL case), while cosine similarity of 0.799 correctly identifies reusable donors.

All primary numbers in this paper are empirically measured on a T4 GPU with bootstrap 95% confidence intervals. Analytical model predictions are clearly labeled and validated against measurements. The PartialAttention mechanism is implemented as three Triton CUDA kernels (scatter, masked projection, partial attention) with 17 passing unit tests. SemBlend is, to our knowledge, the first system unifying embedding-level donor discovery, calibrated recomputation, PartialAttention GPU kernels, and tiered KV storage for semantic KV cache reuse.

A Full Model Specification

Given cosine similarity $s \in [0, 1]$, recomputation ratio $r \in [0, 1]$, and L transformer layers:

$$\text{overlap}(s) = \min(1, s^2) \tag{12}$$

$$\delta(i) = 0.12 + 0.35\, e^{-i/3} + 0.20\, e^{-(L-1-i)/4} \tag{13}$$

$$R = \text{top-}\lfloor rL \rfloor \text{ layers by } \delta \text{ (desc.)} \tag{14}$$

$$W = \frac{1}{L - |R|} \sum_{i \notin R} \delta(i) \tag{15}$$

$$\rho = (1 - \text{overlap}(s)) \cdot W \cdot (1 - r) \tag{16}$$

$$Q(s, r) = 1 - 0.166 \cdot \rho^{0.650} \tag{17}$$

Special case: overlap(1.0) = 1.0 exactly.

B PartialAttention Kernel Specifications

The three Triton kernels use the following grid and block configurations:

Table 14: Triton kernel configurations for PartialAttention.

Kernel	**Grid**	**Block**	**Complexity**
scatter_donor_kv	$(P, 2LH)$	D	$O(P \cdot L \cdot H \cdot D)$
masked_qkv_proj	$(T, 1)$	M×K	$O(T \cdot 3HD \cdot D_m)$
partial_prefill	(C, H)	K	$O(C \cdot T \cdot D)$

P = copy pairs, L = layers, H = heads, D = head dim, T = seq len, C = compute positions, D_m = model dim.

C Reproduction

All code, datasets, and benchmark scripts are available in the project repository. The benchmark CLI accepts dataset selection, endpoint configuration, and output directory parameters. Results are stored as JSON with per-query metrics for independent analysis. Bootstrap confidence intervals use 10,000 resamples with the `scipy.stats.bootstrap` API.

References

[1] Y. Yao *et al.*, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proc. EuroSys*, 2025.

[2] W. Zhang *et al.*, "KVShare: Semantic-Aware KV Cache Sharing for Multi-Tenant LLM Serving," arXiv:2503.16525, 2025.

[3] B. Li *et al.*, "SemShareKV: Semantic-Aware KV Cache Sharing with Token-Level LSH Matching," arXiv:2509.24832, 2025.

[4] S. Wang *et al.*, "ProphetKV: Dual-Stage KV Cache Optimization for RAG with User Query Relevance," arXiv:2602.02579, 2026.

[5] Y. Liu *et al.*, "LMCache: Distributed KV Cache Sharing for Large Language Model Serving," 2025.

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[7] C. Ootomo *et al.*, "CAGRA: Highly Parallel Graph-Based Approximate Nearest Neighbor Search," in *Proc. NeurIPS*, 2024.

[8] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[9] R. Wagner and M. Fischer, "The String-to-String Correction Problem," *J. ACM*, 21(1):168–173, 1974.

[10] W. Kwon *et al.*, "vLLM: Efficient Memory Management for Large Language Model Serving," GitHub, 2023.

[11] Bitext, "Customer Support LLM Chatbot Training Dataset," HuggingFace, 2024.

[12] Z. Bang *et al.*, "GPTCache: An Open-Source Semantic Cache for LLM Applications," GitHub, 2023.

[13] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub, 2023.

[14] P. Tillet *et al.*, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. MAPL*, 2019.